Universidad Tecnológica Nacional
Facultad Regional Córdoba

Lámpara manipulada por comandos de voz

2013

Guazzaroni, Luca

Schamun, Lucas

Sueldo, Enrique

Catedra Informática 2

Lámpara manipulada por comandos de voz

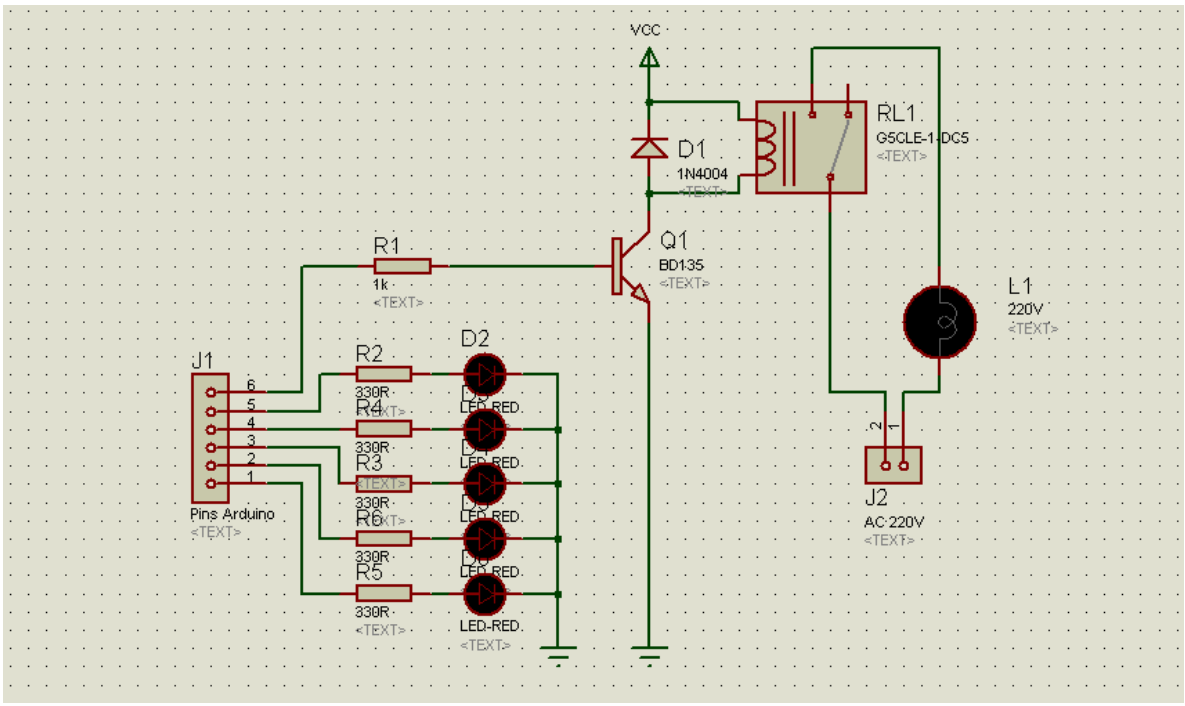
EL presente trabajo corresponde al informe del proyecto final de la materia Informática 2 de la carrera ingeniería electrónica dictada en la Universidad Tecnológica Nacional, Facultad Regional Córdoba. Este proyecto consta de una lámpara que responde a comandos de voz entrenados previamente en un módulo de reconocimiento de voz llamado Easy VR (Easy voice recognition). El cual obtiene señales de audio del ambiente, las procesa y las compara con los comandos grabados en su memoria, y según sea el resultado de ese procesamiento, envía distintos valores a un programa escrito para Arduino en C++ los cuales definirán las acciones que la placa realizara.

Los comandos grabados son los siguientes:

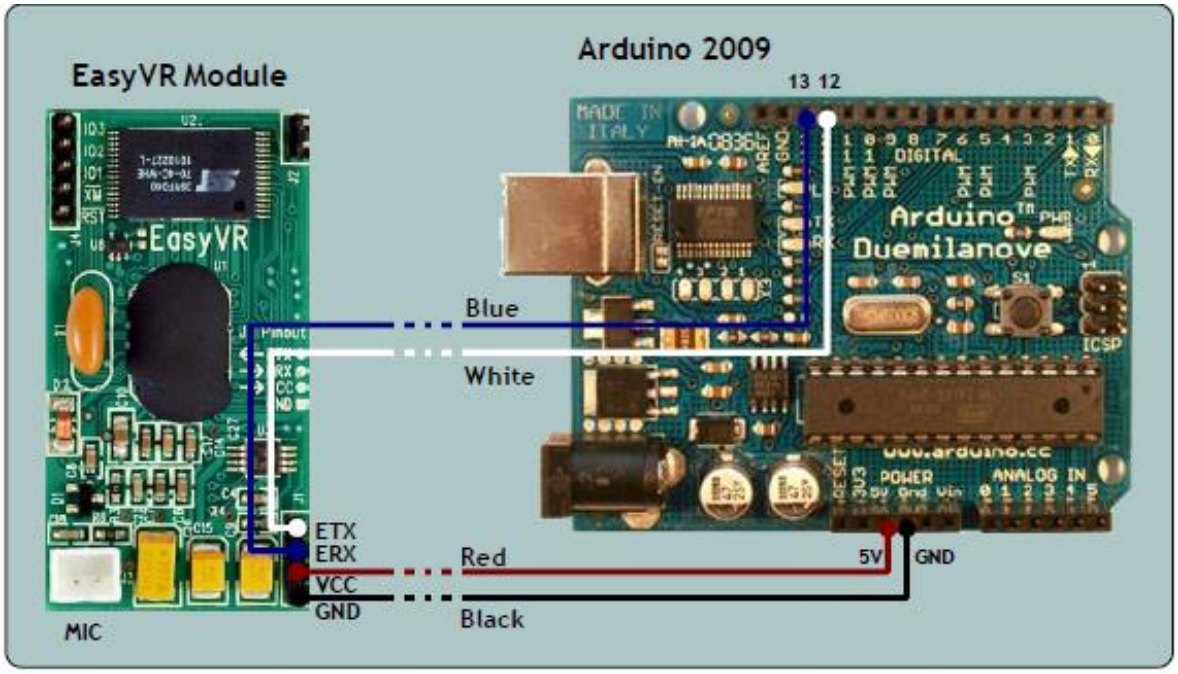
- Hola: prende la lámpara si esta está apagada, en caso de estar prendida no se realizara ninguna acción.
- Chau: Apaga la lámpara si esta prendida, pero en caso de estar apagada no realizara ninguna acción.
- Uno: Hace parpadear la lámpara una vez.
- Dos: Hace parpadear la lámpara dos veces.
- Tres: Hace parpadear la lámpara tres veces.
- Cuatro: Hace parpadear la lámpara cuatro veces.
- Cinco: Hace parpadear la lámpara cinco veces.
- Seis: Hace parpadear la lámpara seis veces.
- Aplauso: Este comando no es la palabra “aplauso” si no que es el sonido de dos aplausos consecutivos. El fin de este comando es prender y apagar la lámpara cuando los comandos “Hola” y “Chau” no respondan, por ejemplo, porque hay mucho ruido ambiente. Si se aplaude de la manera determinada, la lámpara se apagara si esta prendida y se encenderá si está apagada.

El proyecto está formado por una placa Arduino, un módulo Easy VR, una placa con leds, una placa con un relé, la lámpara y los gabinetes.

El esquemático de la placa de leds y la placa del relé es el siguiente (J1 corresponde a los pines del Arduino que va conectados a estas placas):



La conexión del módulo Easy VR con Arduino es la siguiente:



Funcionamiento del proyecto

El proyecto funciona de la siguiente manera:

- Al energizar el Arduino con 12V DC, comienza el programa guardado en él. Al realizarse el setup (configuración del hardware), Arduino verifica que la conexión con el Easy VR es correcta, haciendo esta verificación hasta que el modulo es detectado. Una vez detectado el Easy VR los leds parpadearan indicando que el modulo ha sido reconocido por Arduino.

- Una vez detectado el Easy VR, este procede a escuchar los sonidos del ambiente y en caso de detectar algo, analizarlo comparándolo con los comandos guardados en su memoria.

- Si el Easy VR detecta una señal de audio que coincide con un comando grabado, los cinco leds ubicados en el gabinete destellaran dos veces indicando que el reconocimiento fue exitoso. Luego, según sea el comando, Arduino pondrá en 1 o en 0 el pin definido como “rele” el cual es el encargado de energizar la base un transistor que funciona como switch, energizando o quitándole corriente a los terminales de la bonina de un relé que está conectado a una lámpara de 220V.

- Si en el momento en el que el Easy VR está detectando las señales de audio del ambiente no se percibe ninguna señal, se producirá un timeout lo cual hará que dos leds de la placa destellen dos veces.

- Si el Easy VR detecto alguna señal de audio del entorno, pero esta no coincide con ninguno de los comandos grabados, se producirá un error lo cual hará que un led de la placa de leds destelle dos veces.

Arduino

¿Qué es Arduino?

Arduino es una plataforma open source para la creación de prototipos basada en software y hardware flexibles y fáciles de usar. Se creó para artistas, diseñadores, aficionados y cualquiera interesado en crear entornos u objetos interactivos.

Arduino puede tomar información del entorno a través de sus pines de entrada de toda una gama de sensores y puede afectar aquello que le rodea controlando luces, motores y otros actuadores. El microcontrolador en la placa Arduino se programa mediante el lenguaje de programación Arduino (que básicamente es C/C++) a través del entorno de desarrollo Arduino que puede ejecutarse en plataformas como Linux, Windows y MacOS X gracias a que está desarrollado en Java y cuya compilación parte de un compilador de AVR. Como este IDE está basado en el ambiente de desarrollo de Processing, permite la creación de sketchs visuales lo cual tiene como objetivo brindar una plataforma amigable y fácil de usar, permitiendo realizar proyectos en un solo ambiente de programación, en un lenguaje propio o utilizando C++. La mayor virtud del entorno de desarrollo de Arduino es que una vez terminado el código, permite cargarlo directamente a la placa mediante una conexión USB, lo que permite crear y probar rápidamente los distintos códigos desarrollados en la placa sin necesidad de utilizar herramientas adicionales. Esto, sumado a las distintas librerías que ofrecen funciones que encapsulan muchos detalles complejos del manejo del hardware haciendo fácil el manejo de la placa, hacen de Arduino una plataforma potente y fácil de utilizar.

Arduino Mega

El Arduino Mega es una placa microcontrolador basada el ATmeg1280. Tiene 54 entradas/salidas digitales (de las cuales 14 proporcionan salida PWM), 16 entradas analógicas, 4 UARTS (puertos serie por hardware), un cristal oscilador de 16MHz, conexión USB, entrada de corriente, 2 conectores ICSP y botón de reset. Contiene todo lo necesario para hacer funcionar el microcontrolador y simplemente se debe conectar al ordenador mediante el cable USB o alimentarlo mediante cualquier fuente de corriente continua.

Resumen de prestaciones:

Microcontrolador	ATmega1280
Voltaje de funcionamiento	5V
Voltaje de entrada (recomendado)	7-12V
Voltaje de entrada (limite)	6-20V
Pines E/S digitales	54 (14 proporcionan salida PWM)
Pines de entrada analógica	16
Intensidad por pin	40 mA
Intensidad en pin 3.3V	50 mA
Memoria Flash	128 KB de las cuales 4 KB las usa el gestor de arranque(bootloader)
SRAM	8 KB
EEPROM	4 KB
Velocidad de reloj	16 MHz

Alimentación

El Arduino Mega puede ser alimentado vía la conexión USB o con una fuente de alimentación externa de corriente continua. El origen de la alimentación se selecciona automáticamente. En el caso de alimentar la placa con un transformador se lo puede conectar usando un conector macho de 2.1mm con centro positivo en el conector hembra de la placa, Si se utiliza una batería se la puede conectar mediante cables a los pines Gnd y Vin.

La placa puede trabajar con una alimentación externa de entre 6 a 20 voltios. Si el voltaje suministrado es inferior a 7V el pin de 5V puede proporcionar menos de 5 Voltios y la placa puede volverse inestable, si se usan más de 12V los reguladores de voltaje se pueden sobrecalentar y dañar la placa. El rango recomendado es de 7 a 12 voltios.

Los pines de alimentación son los siguientes:

- **VIN:** La entrada de voltaje a la placa Arduino cando se está usando una fuente externa de alimentación (en opuesto a los 5 voltios de la conexión USB). Se puede

proporcionar voltaje a través de este pin, o, si se está alimentado a través de la conexión de 2.1mm, acceder a ella a través de este pin.

- **5V.** Es la fuente de voltaje estabilizada usada para alimentar el microcontrolador y otros componentes de la placa. Esta puede provenir de VIN a través de un regulador integrado en la placa, o proporcionada directamente por el USB u otra fuente estabilizada de 5V.
- **3V3:** Una fuente de voltaje a 3.3 voltios generada en el chip FTDI integrado en la placa. La corriente máxima soportada por este pin es de 50mA.
- **GND:** Pines de toma de tierra.

Memoria

El ATmega1280 tiene 128KB de memoria flash para almacenar código de los cuales 4KB son usados para el arranque del sistema, 8 KB de memoria SRAM y 4KB de memoria EEPROM, a la cual se puede acceder (para leer o escribir) mediante funciones pertenecientes a la librería EEPROM que está incluida en las librerías del entorno de desarrollo.

Entradas y Salidas

Cada uno de los 54 pines digitales del Arduino Mega pueden utilizarse como entradas o como salidas usando las funciones pinMode(), digitalWrite(), y digitalRead() operando a 5 voltios. Cada pin puede proporcionar o recibir una intensidad máxima de 40mA y tiene una resistencia interna (desconectada por defecto)de 20-50kOhms.

Algunos pines tienen funciones especializadas:

- **Comunicación Serie**: Se utilizan para recibir (RX) y transmitir (TX) datos a través de puerto serie TTL. En el Arduino Mega se cuenta con cuatro puertos seriales por defecto (ya que mediante la librería SoftWareSerial se puede configurar cualquier par de pines digitales como pines de comunicación serie). Puertos serie:
    - Serie: pin 0 (RX) y pin 1 (TX) que a su vez están conectados con el USB.
    - Serie1: pin 19 (RX) y pin 18 (TX).
    - Serie2: pin 17 (RX) y pin 16(TX).
    - Serie3: pin 15 (RX) y pin 14 (TX).

- **Interrupciones Externas:** Estos pines se pueden configurar para lanzar una interrupción en un valor LOW (0V), en flancos de subida o bajada (cambio de LOW a HIGH (5V) o viceversa), o en cambios de valor a través de la función attachInterrupt. pin 2 (interrupción 0), pin 3 (interrupción 1), pin 18 (interrupción 5), pin 19 (interrupción 4), pin 20 (interrupción 3), y pin 21 (interrupción 2).
- **PWM:** Los pines del 0 al 13 proporcionan una salida PWM (Pulse Wave Modulation, modulación de onda por pulsos) de 8 bits de resolución (valores de 0 a 255) a través de la función analogWrite().
- **SPI:** Pin 50 (SS), pin 51 (MOSI), pin 52 (MISO) y pin 53 (SCK), estos pines proporcionan comunicación SPI, que a pesar de que el hardware la proporcione actualmente no está incluido en el lenguaje Arduino.
- **LED:** 13. Hay un LED integrado en la placa conectado al pin digital 13, cuando este pin tiene un valor HIGH(5V) el LED se enciende y cuando este tiene un valor LOW(0V) este se apaga.

El Mega tiene 16 entradas analógicas, y cada una de ellas proporciona una resolución de 10bits (1024 valores). Por defecto se mide de tierra a 5 voltios, aunque es posible cambiar la cota superior de este rango usando el pin AREF y la funciónanalogReference(). Además algunos pines tienen funciones especializadas:

- **I²C:** Pin 20 (SDA) y pin 21 (SCL). Soporte del protocolo de comunicaciones I²C (TWI) usando la librería Wire.

También hay otros pines en la placa:

- **AREF:** Voltaje de referencia para la entradas analógicas usado analogReference().
- **Reset:** Suministrar un valor LOW (0V) para reiniciar el microcontrolador. Típicamente usado para añadir un botón de reset a los shields que no dejan acceso a este botón en la placa.

Comunicaciones

EL Arduino Mega facilita en varios aspectos la comunicación con el ordenador, otro Arduino u otros microcontroladores. El ATmega1280 proporciona cuatro puertos de comunicación vía serie UART TTL (5V), un chip FTDI FT232RL integrado en la placa (que canaliza esta comunicación serie a través del USB) y los drivers FTDI incluidos en el software de Arduino que proporcionan un puerto serie virtual en el ordenador. El software (el entorno de desarrollo) incluye un monitor de puerto serie que permite enviar y recibir información

textual con la placa Arduino. Los LEDS RX y TX de la placa parpadearan cuando se detecte comunicación transmitida través del chip FTDI y la conexión USB (no parpadearan si se usa la comunicación serie a través de los pines 0 y 1). La librería SoftwareSerial permite comunicación serie por cualquier par de pines digitales del Mega. El ATmega1280 también soporta la comunicación I2C (TWI) y SPI. El software de Arduino incluye una librería Wire para simplificar el uso el bus I2C.

Programación

El Arduino Mega se puede programar con el software Arduino ya que el microcontrolador ATmega1280 incorporado en el Arduino Mega viene precargado con un gestor de arranque (bootloader) que permite cargar nuevo código sin necesidad de un programador por hardware externo, comunicándose utilizando el protocolo STK500. También se puede saltar el gestor de arranque y programar directamente el microcontrolador a través del puerto ISCP (In Circuit Serial Programming).

Reinicio Automatico por Software

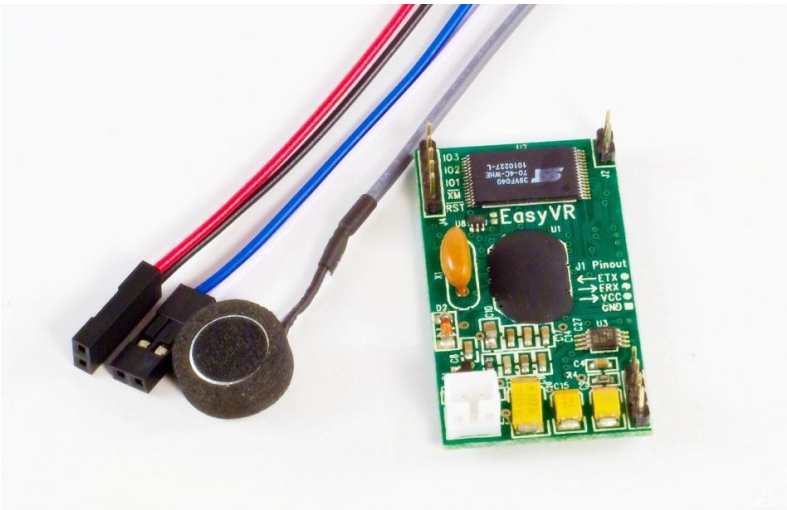
En vez de necesitar reiniciar presionando físicamente el botón de reset antes de cargar, el Arduino Mega está diseñado de manera que es posible reiniciar por software desde el ordenador donde al cual este conectado. Una de las líneas de control de flujo (DTR) del FT232RL está conectada a la línea de reinicio del ATmega1280 a través de un condensador de 100 nanofaradios. Cuando la línea se pone a LOW (0V), la línea de reinicio también se pone a 0V el tiempo suficiente para reiniciar el chip. El software de Arduino utiliza esta característica para permitir cargar los sketches con solo apretar un botón del entorno. Dado que el gestor de arranque tiene un lapso de tiempo para ello, la activación del DTR y la carga del sketch se coordinan perfectamente.

Protección contra sobretensiones en USB

El Arduino Mega tiene un multifusible reinicializable que protege la conexión USB del ordenador de cortocircuitos y sobretensiones. A aparte que la mayoría de ordenadores proporcionan su propia protección interna, el fusible proporciona una capa extra de protección. Si más de 500mA son detectados en el puerto USB, el fusible automáticamente corta la conexión hasta que el cortocircuito o la sobretensión hayan desaparecido.

Easy VR

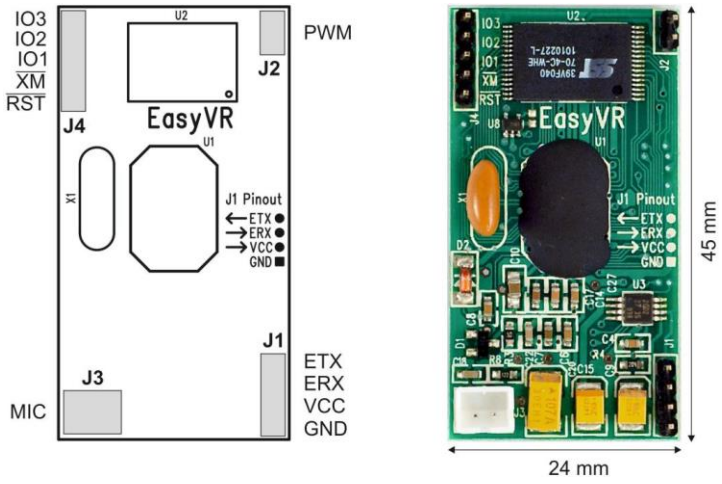
El Easy VR es un potente modulo para el reconocimiento de voz y reproducción de sonidos. Las voces a reconocer pueden estar predefinidas y grabadas internamente (voces independientes del micrófono SI) o pueden estar determinadas por el usuario mediante el entrenamiento de la placa a través de software.



Características:

- 26 comandos y voces pre-programados (SI) en: en inglés, italiano, japonés, alemán, francés y español
- Admite hasta 32 comandos y sus correspondientes voces definidas por el usuario (SD) en cualquier idioma
- Los comandos y voces SD se pueden organizar en hasta 16 grupos
- Se dispone de un interface gráfico de usuario (GUI) para Windows que permite una rápida familiarización con el uso del módulo.
- El módulo se puede conectar fácilmente con cualquier tipo de controlador mediante una sencilla comunicación serie.
- Dispone de un potente protocolo de comandos para el desarrollo de cualquier aplicación basada en el reconocimiento de voz.
- Alimentación de 3.3V a 5V
- 3 líneas de E/S o GPIO (IO1, IO2 e IO3) que se controlan directamente mediante los comandos correspondientes.
- Salida PWM de audio que admite la conexión con un altavoz de 8Ω.
- Capacidad de reproducción de hasta 9 minutos de sonidos y/o voces.

Datos Técnicos



Connector	Number	Name	Type	Description
J1	1	GND	-	Ground
	2	VCC	I	Voltage DC input
	3	ERX	I	Serial Data Receive (TTL level)
	4	ETX	O	Serial Data Transmit (TTL level)
J2	1-2	PWM	O	Differential audio output (can directly drive 8Ω speaker)
J3	1	MIC_RET	-	Microphone reference ground
	2	MIC_IN	I	Microphone input signal
J4	1	/RST	I	Active low asynchronous reset (internal 100K pull-up)
	2	/XM	I	Boot select (internal 1K pull-down)
	3	IO1	I/O	General purpose I/O (**3.0 VDC** TTL level)
	4	IO2	I/O	General purpose I/O (**3.0 VDC** TTL level)
	5	IO3	I/O	General purpose I/O (**3.0 VDC** TTL level)

Interface serie

El módulo EasyVR se comporta como un dispositivo esclavo que se controla desde un dispositivo Master o controlador, mediante una comunicación serie asíncrona tipo UART con las siguientes características:

- Velocidad de comunicación: 9600 baudios (por defecto), 19200, 38700, 57600 y 115200 baudios.
- Trama: 8 bits de datos, 1 bit de stop y sin paridad.

Junto con el módulo EasyVR se adjunta un software para Windows, el EasyVR Commander. Se trata de un interface gráfico de usuario (GUI) que permite utilizar y analizar de forma rápida, eficaz y sencilla todas las funciones del módulo de reconocimiento EasyVR.

Comunicación con un Host

El EasyVR se puede conectar, además de a un PC, a cualquier otro sistema de control basado en microcontroladores como pueden ser los PIC's, Basic Stamp, Atmel, Arduino, etc. que actuarán como Host.

El EasyVR emplea un UART como interface estándar de comunicación con niveles lógicos compatibles TTL de acuerdo a la tensión con la que se le alimente (3.3-5V). Las conexiones entre el EasyVR y el microcontrolador principal (Host MCU) son compatibles a nivel eléctrico. Lo único que se debe tener en cuenta es que la señal ETX de transmisión de datos del EasyVR debe conectarse con la de recepción RX del controlador y viceversa. Es decir, las señales se deben cruzar. Por defecto, al conectar la alimentación, la configuración de la comunicación es de 9600 baudios, 8 bits de datos, 1 de stop y sin paridad. Los baudios pueden configurarse en el rango de 9600 hasta 115200.

- En la comunicación solo se emplean caracteres ASCII imprimibles, lo que permite el control de EasyVR desde cualquier software de comunicación para PC como, por ejemplo, el hyperterminal de Windows. Se siguen los siguientes criterios:
- Los comandos que se trasmiten desde el Host al módulo por la línea TX son siempre caracteres en minúsculas. Por su parte el EasyVR responde al Host por la línea RX unos caracteres de estado que también son siempre en minúsculas. Los argumentos o parámetros necesarios tanto para los comandos transmitidos como los de estado recibidos, son siempre caracteres en mayúsculas.
- Cada comando que transmite el Host por la línea TX con los argumentos o parámetros necesarios (cero o más caracteres), genera la correspondiente respuesta por parte del EasyVR que se recibe por la línea RX. Esta respuesta también puede tener parámetros o argumentos de cero o más caracteres.
- Existe un retardo mínimo entre cada byte enviado al Host por parte del EasyVR, que por defecto es de 20mS. Se puede configurar en rangos que van de 0 a 9 ms, de 10 a 90 ms y de 100mS a 1 s. Esto permite adaptarse a la velocidad de proceso alta o baja por parte del Host.

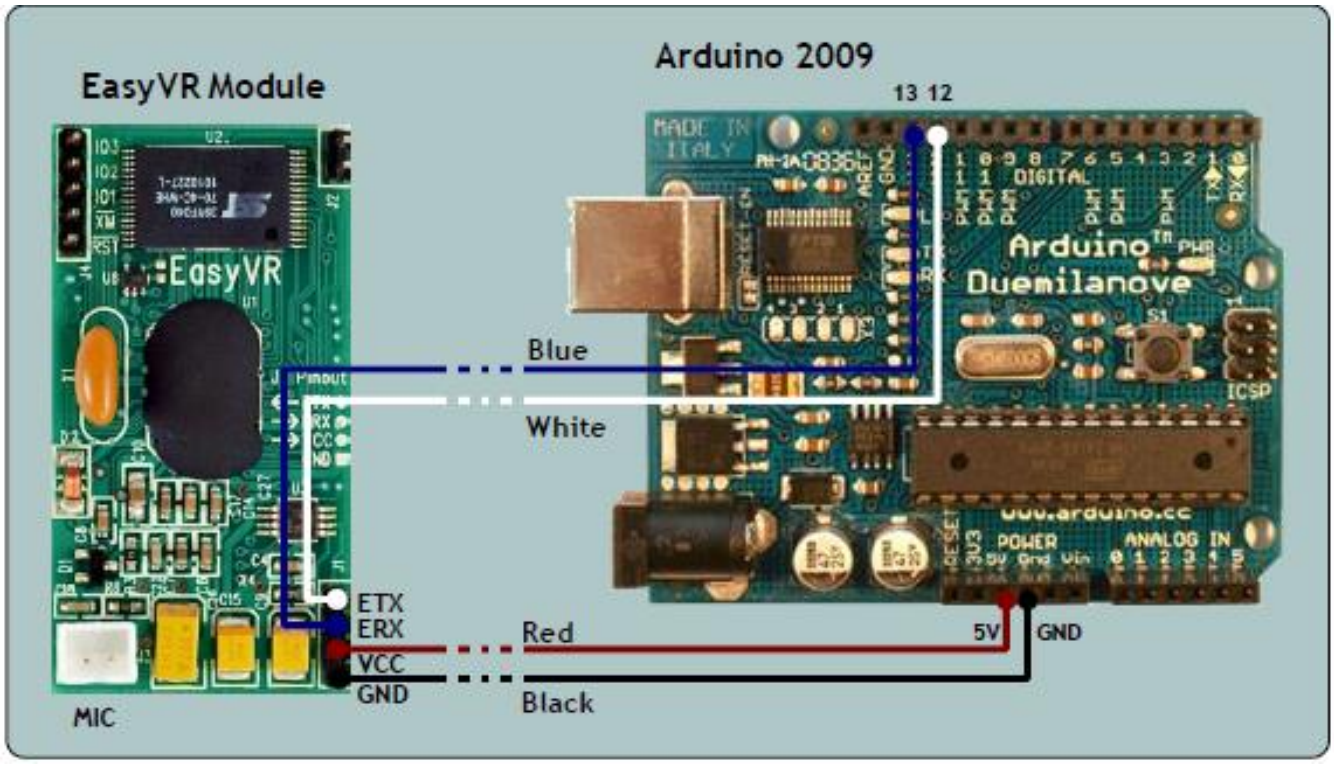
- El flujo de información lo controla el Host en todo momento. Cada byte que manda el EasyVR como réplica a un determinado comando, debe ser reconocido por parte del Host que le envía el carácter espacio (0x20) para recibir así los sucesivos bytes de respuesta si los hubiera.
- Los comandos y/o argumentos incorrectos quedan reflejados por el correspondiente byte de estado que envía el EasyVR al Host para su oportuno tratamiento.
- Por otra parte, si el Host no envía todos los bytes de argumentos o parámetros necesarios por un determinado comando, este es ignorado por parte del módulo EasyVR.
- Cada vez que se conecta la alimentación el módulo EasyVR queda en el modo sleep de bajo consumo. Para iniciar la comunicación basta con que el Host le mande cualquier carácter.

Conexión Easy VR con Arduino

Básicamente se puede conectar el módulo Easy VR con la placa Arduino de dos formas:

Modo Puente

Permite manejar el modulo usando la librería de software serial y cargar ambas placas mediante la PC sin cambiar la configuración de los pines.

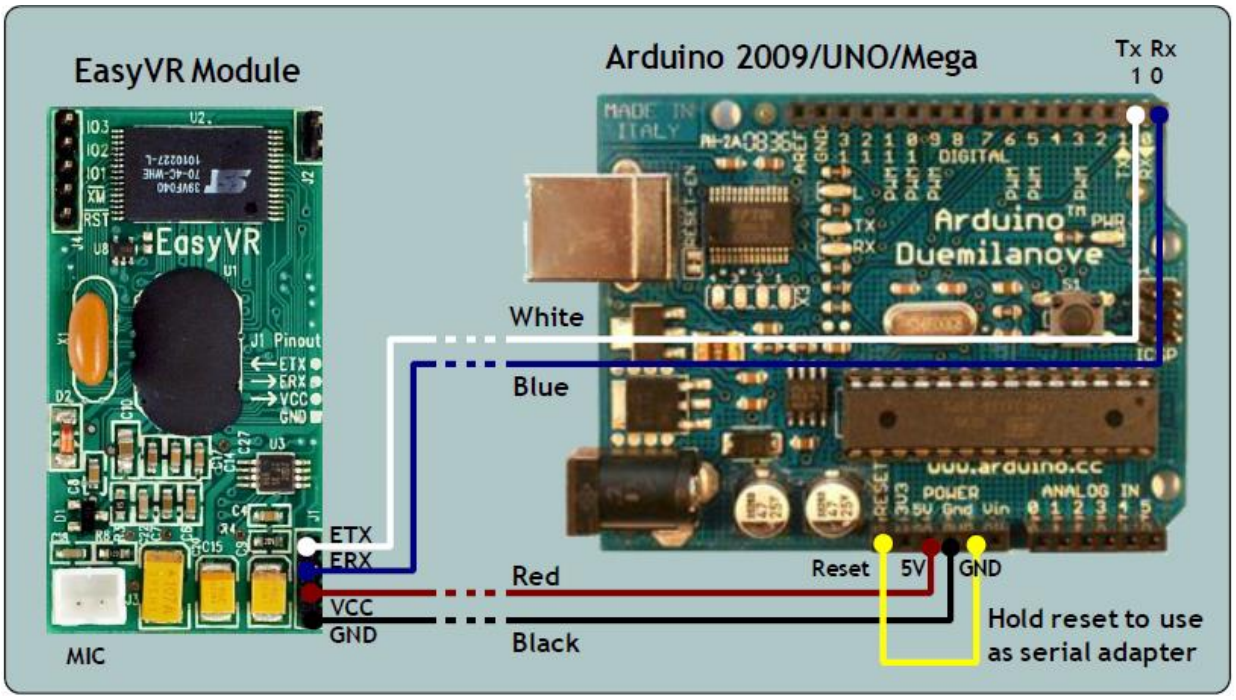


Este es mejor modo para realizar aplicaciones con el Easy VR ya que nos provee una simple comunicación entre el microcontrolador de Arduino y la PC. Además, todos los ejemplos provistos por Arduino manejan el modo puente automáticamente cuando el Easy VR requiere una conexión.

Aquí el Easy VR y Arduino intercambian datos a través de un puerto serial creado mediante la librería “SoftwareSerial” y a su vez se pueden enviar datos a través del puerto USB a la PC para analizar el correcto funcionamiento del programa cargado al Arduino.

Modo adaptador

Se utiliza el Arduino como un adaptador USB/Serial manteniendo en reset al microcontrolador pero esta configuración solo sirve para grabar los comandos de voz en la memoria interna del Easy VR a través del Software apropiado.



En esta configuración se conecta el pin “reset” con un pin de toma a tierra, manteniendo en estado de “reset” al microcontrolador de Arduino haciendo que este último funcione como un adaptador entre al Easy VR y la PC que a través del programa EasyVR commander cargara al módulo los patrones de voz grabados por el usuario. Aquí se conectan el trasmisor de datos (TX) y el receptor de datos (RX) con el puerto serial del Arduino que están conectados al puerto USB y por lo tanto permite el traspaso de datos entre el modulo y la PC.

Programa

```
#if defined(ARDUINO) && ARDUINO >= 100
   #include "Arduino.h"
        #include "SoftwareSerial.h"
        SoftwareSerial port(12,13);
#else
        #include "WProgram.h"
        #include "NewSoftSerial.h"
        NewSoftSerial port(12,13);
#endif

#include "EasyVR.h"

EasyVR easyvr(port);

enum Groups
{
  GROUP_1  = 1,
};

enum Group1
{
  G1_HOLA = 0,
  G1_UNO = 1,
  G1_DOS = 2,
  G1_TRES = 3,
  G1_CUATRO = 4,
  G1_CINCO = 5,
  G1_SEIS = 6,
  G1_CHAU = 7,
  G1_APLAUSO = 8,
};

EasyVRBridge bridge;

int8_t group, idx;
int    leds[5]={45, 47, 49, 51, 53};
int    rele=40;
int    estado=0;
int    i, j;

void prende(int);
void apaga(int);
void parpadea();
void action();
```

```
void setup()
{
  // ---- Setup de los pin de los leds ---- //
  for(i=0 ;i<5 ;i++)
  {
     pinMode(leds[i], OUTPUT);
  }

  // ---- Setup del pin del rele ---- //
  pinMode(rele, OUTPUT);

  // ---- Setup del Easy VR ---- //
  if (bridge.check())
  {
     cli();
     bridge.loop(0, 1, 12, 13);
  }

  port.begin(9600);

  while(!easyvr.detect())
  {
     delay(100);
  }

  easyvr.setPinOutput(EasyVR::IO1, LOW);
  prende(5);
  delay(200);
  parpadea();
  easyvr.setTimeout(5);
  easyvr.setLanguage(0);

  group = EasyVR::TRIGGER;
}

void loop()
{
  group=GROUP_1;
  easyvr.setPinOutput(EasyVR::IO1, HIGH); // El Easy VR empieza a "escuchar"
  easyvr.recognizeCommand(group);

  do
  {
  }
  while (!easyvr.hasFinished());

  easyvr.setPinOutput(EasyVR::IO1, LOW); // LED off
```

```
  idx = easyvr.getWord();
  if (idx >= 0)
  {
    return;
  }

  idx = easyvr.getCommand();
  if (idx >= 0)
  {
   easyvr.playSound(0, EasyVR::VOL_FULL);

   action();
  }
  else
  {
      if (easyvr.isTimeout())
      {
                    prende(1);
                    apaga(1);
      }
      int16_t err = easyvr.getError();
      if (err >= 0)
      {
                    prende(2);
                    apaga(2);
      }
  }
}

void prende(int num)
{
   for(i=0 ;i<num ;i++)
   {
      digitalWrite(leds[i], HIGH);
      delay(80);
   }
}

void apaga(int num)
{
   for(i=0 ;i<num;i++)
   {
      digitalWrite(leds[i], LOW);
   }
}
```

```
void parpadea()
{
   for(j=0;j<2;j++)
   {
    for(i=0 ;i<5 ;i++)
    {
       digitalWrite(leds[i], HIGH);
    }
    delay(150);

    for(i=0 ;i<5 ;i++)
    {
       digitalWrite(leds[i], LOW);
    }
    delay(150);
   }
}

void ReleParpadea(int num)
{
  if(estado==0)
  {
     for(i=0 ;i<num ;i++)
     {
          digitalWrite(rele, HIGH);
          delay(500);
          digitalWrite(rele, LOW);
          delay(500);
      }
   }
   else
   {
      for(i=0 ;i<num ;i++)
     {
          digitalWrite(rele, LOW);
          delay(500);
          digitalWrite(rele, HIGH);
          delay(500);
      }
   }
}
```

```
void action()
{
  switch (group)
  {
    case GROUP_1:
      parpadea();

      switch (idx)
      {
        case G1_HOLA:
          digitalWrite(rele, HIGH);
          estado=1;
        break;

                        case G1_UNO:
          ReleParpadea(1);
        break;

                        case G1_DOS:
          ReleParpadea(2);
        break;

                        case G1_TRES:
          ReleParpadea(3);
        break;

                        case G1_CUATRO:
          ReleParpadea(4);
        break;

                        case G1_CINCO:
          ReleParpadea(5);
        break;

                        case G1_SEIS:
          ReleParpadea(6);
        break;

                        case G1_CHAU:
          digitalWrite(rele, LOW);
          estado=0;
        break;

                        case G1_APLAUSO:
          if(estado==0)
          {
            digitalWrite(rele, HIGH);
            estado=1;
```

```
            }
            else
            {
               digitalWrite(rele, LOW);
               estado=0;
            }
           break;
          }
      break;
   }
}
```

SI se quiere analizar el comportamiento del proyecto mediante la PC se le tiene que agregar la comunicación serial del Arduino con la PC a través de Serial.Begin(9600) y pasar los datos a través de Serial.Println(datos). Hicimos esto para analizar cuales palabras reconocía mejor el modulo y como grabarlas para que puedan ser dichas por varias personas.

```
#if defined(ARDUINO) && ARDUINO >= 100
        #include "Arduino.h"
        #include "SoftwareSerial.h"
        SoftwareSerial port(12,13);
#else
        #include "WProgram.h"
        #include "NewSoftSerial.h"
        NewSoftSerial port(12,13);
#endif

#include "EasyVR.h"

EasyVR easyvr(port);

enum Groups
{
        GROUP_1  = 1,
};

enum Group1
{
        G1_HOLA = 0,
        G1_UNO = 1,
        G1_DOS = 2,
        G1_TRES = 3,
        G1_CUATRO = 4,
        G1_CINCO = 5,
        G1_SEIS = 6,
        G1_CHAU = 7,
        G1_APLAUSO = 8,
};
```

```
EasyVRBridge bridge;

int8_t group, idx;
int    leds[5]={45, 47, 49, 51, 53};
int    rele=40;
int    estado=0;
int    i, j;

void prende(int);
void apaga(int);
void parpadea();
void action();

void setup()
{
 // ---- Setup de los pin de los leds ---- //
 for(i=0 ;i<5 ;i++)
 {
    pinMode(leds[i], OUTPUT);
 }

 // ---- Setup del pin del rele ---- //
 pinMode(rele, OUTPUT);

 // ---- Setup del Easy VR ---- //
 if (bridge.check())
 {
          cli();
          bridge.loop(0, 1, 12, 13);
 }

 Serial.begin(9600);
 port.begin(9600);

 while(!easyvr.detect())
 {
           Serial.println("Easy VR NO detectado!!!");
   delay(100);
 }

 easyvr.setPinOutput(EasyVR::IO1, LOW);
 Serial.println("Easy VR detectado");
 prende(5);
 delay(200);
 parpadea();
 easyvr.setTimeout(5);
 easyvr.setLanguage(0);
```

```
  group = EasyVR::TRIGGER;
}

void loop()
{
  group=GROUP_1;
  easyvr.setPinOutput(EasyVR::IO1, HIGH); // El Easy VR empieza a "escuchar"
  Serial.print("Diga un comando del grupo: ");
  Serial.println(group);
  easyvr.recognizeCommand(group);

  do
  {
  }
  while (!easyvr.hasFinished());

  easyvr.setPinOutput(EasyVR::IO1, LOW); // LED off

  idx = easyvr.getWord();
  if (idx >= 0)
  {
    return;
  }
  idx = easyvr.getCommand();
  if (idx >= 0)
  {

    uint8_t train = 0;
    char name[32];
    Serial.print("Comando: ");
    Serial.print(idx);
    if (easyvr.dumpCommand(group, idx, name, train))
    {
      Serial.print(" = ");
      Serial.println(name);
    }
    else
      Serial.println();
    easyvr.playSound(0, EasyVR::VOL_FULL);

    action();
  }
  else
  {
    if (easyvr.isTimeout())
    {
```

```
        Serial.println("Tiempo de espera agotado, intente de nuevo...");
        prende(1);
        apaga(1);
     }
    int16_t err = easyvr.getError();
    if (err >= 0)
    {
      Serial.print("Error ");
      Serial.println(err, HEX);
      prende(2);
      apaga(2);
    }
  }
}

void prende(int num)
{
   for(i=0 ;i<num ;i++)
   {
      digitalWrite(leds[i], HIGH);
      delay(80);
   }
}

void apaga(int num)
{
   for(i=0 ;i<num;i++)
   {
      digitalWrite(leds[i], LOW);
   }
}

void parpadea()
{
   for(j=0;j<2;j++)
   {
     for(i=0 ;i<5 ;i++)
     {
        digitalWrite(leds[i], HIGH);
     }
     delay(150);

     for(i=0 ;i<5 ;i++)
     {
        digitalWrite(leds[i], LOW);
     }
     delay(150);
   }
```

```
}

void ReleParpadea(int num)
{
  if(estado==0)
  {
     for(i=0 ;i<num ;i++)
     {
          digitalWrite(rele, HIGH);
          delay(500);
          digitalWrite(rele, LOW);
          delay(500);
      }
   }
   else
   {
      for(i=0 ;i<num ;i++)
     {
          digitalWrite(rele, LOW);
          delay(500);
          digitalWrite(rele, HIGH);
          delay(500);
      }
   }
}


void action()
{
   switch (group)
   {
     case GROUP_1:
        parpadea();

        switch (idx)
        {
          case G1_HOLA:
             digitalWrite(rele, HIGH);
             estado=1;
           break;

                              case G1_UNO:
           ReleParpadea(1);
           break;

                              case G1_DOS:
           ReleParpadea(2);
           break;
```

```
                        case G1_TRES:
            ReleParpadea(3);
           break;

                        case G1_CUATRO:
            ReleParpadea(4);
           break;

                        case G1_CINCO:
            ReleParpadea(5);
           break;

                        case G1_SEIS:
            ReleParpadea(6);
           break;

                        case G1_CHAU:
            digitalWrite(rele, LOW);
            estado=0;
           break;

                        case G1_APLAUSO:
            if(estado==0)
            {
               digitalWrite(rele, HIGH);
               estado=1;
            }
            else
            {
               digitalWrite(rele, LOW);
               estado=0;
            }
           break;
         }
      break;
   }
}
```

Análisis del Programa

A continuación analizaremos parte por parte el programa cargado al Arduino, de manea que se logre una correcta comprensión del mismo y de la interacción del Easy VR con el Arduino.

```
#if defined(ARDUINO) && ARDUINO >= 100
 #include "Arduino.h"
 #include "SoftwareSerial.h"
 SoftwareSerial port(12,13);
#else
 #include "WProgram.h"
 #include "NewSoftSerial.h"
 NewSoftSerial port(12,13);
#endif
```

Lo que se busca con estos condicionales es corroborar con que versión del compilador de Arduino se está trabajando ya que depende cual sea, las bibliotecas que nos son de utilidad para el manejo del Easy VR son distintas. Cada compilador Arduino tiene por defecto una constante de nombre "ARDUINO" que depende del número de versión del mismo, por ejemplo:

Versión 0022 -> ARDUINO=22
Versión 0023 -> ARDUINO=23
Versión 1.0 -> ARDUINO=100
Versión 1.05 -> ARDUINO=105
Versión 1.5 -> ARDUINO=150

Se debe hacer esto ya las primeras versiones traían la biblioteca "WProgram.h" que contiene las funciones elementales del Arduino y "SoftwareSerial.h" que se utiliza para configurar parejas de pines del Arduino como puertos de comunicación serial. Como posteriormente esta última se vio superada por una nueva biblioteca "NewSoftSerial.h", se recomendó que para realizar acciones de comunicación serial convenía usar esta nueva librería que (en el caso de las versiones anteriores a la 1.0 del compilador Arduino) tiene que ser descargada de http://arduiniana.org/libraries/newsoftserial/ y moverla a la carpeta "libraries" que se encuentra en la carpeta del compilador Arduino en C:\Program Files (x86) (o en donde se lo haya guardado al instalarlo).
A partir de la versión 1.0 la biblioteca "WProgram.h" sumo algunas mejoras y fue renombrada "Arduino.h" mientras que "NewSoftSerial.h" reemplazo a "SoftwareSerial.h" y tomo su nombre.

La línea "SoftwareSerial port(12,13)" hace que los pines 12 y 13 permitan la comunicación serial siendo el 12 quien recibe datos (RX) y 13 quien transmite (TX). Por ende el TX (transmisor de datos) del Easy VR ira conectado al pin 12 (receptor de Arduino) y el RX (receptor de datos) del Easy VR ira conectado al pin 13 (trasmisor de Arduino).En nuestro caso (como usamos Arduino mega que tiene 3 tres puertos serie adicionales) podríamos prescindir de esta línea de código y conectar el Easy VR a los pines

(TX con 19, RX con 18) e inicializar Serial1.begin(9600);
(TX con 17, RX con 16) e inicializar Serial2.begin(9600);
(TX con 15, RX con 14) e inicializar Serial1.begin(9600);

pero esto haría que el programa no sirviera para las otras placas Arduino(uno, Leonardo, mini, etc.)

```
#include "EasyVR.h"
```

Incluye la biblioteca “EasyVR.h” que contiene una clase llamada EasyVR y otra llamada EasyVRBridge que contienen funciones para el manejo del reconocedor voz (escuchar, comparar comando, etc) y para verificar que la conexion entre Arduino y el Easy VR es correcta.

```
EasyVR easyvr(port);
```

Crea un objeto "easyvr" de la clase EasyVR (definida en EasyVR.h) y se le pasa la variable port (puerto serie declarado con SoftwareSerial(12,13)) para detectar el Easy VR , despertarlo y verificar si reponde correctamente.

```
enum Groups
{
 GROUP_1  = 1,
};

enum Group1
{
 G1_HOLA = 0,
 G1_UNO = 1,
 G1_DOS = 2,
 G1_TRES = 3,
 G1_CUATRO = 4,
 G1_CINCO = 5,
 G1_SEIS = 6,
 G1_CHAU = 7,
 G1_APLAUSO = 8,
};
```

Enumera todos los grupos y todos los comandos pertenecientes a cada grupo con sus respectivos valores.

```
EasyVRBridge bridge;
```

Declara un objeto de nombre bridge de la clase EasyVRBridge (definida en la librería EasyVR.h). EL fin de este objeto cuyas funciones miembro son del tipo bool es determinar si la conexión entre el EasyVR y el Arduino ha sido solicitada.

```
int8_t group, idx;
```

Declara dos variables del tipo “int8_t”, que no son más que dos integers de 8 bits con signo. Se deben declarar así y no como “int” ya que las funciones en las que utilizaremos estas variables tienen parámetros definidos como “int8_t”.

```
int   leds[5]={45, 47, 49, 51, 53};
int   rele=40;
int   estado=0;
int   i, j;
```

Declara que numero de pines van a hacer usados para cada tarea y se les designa un nombre para mejorar la legibilidad del código.

```
void prende(int);
void apaga(int);
void parpadea();
```

Se declara las funciones que van a ser las encargadas de generar los diferentes efectos con los leds.

```
void ReleParpadea(int );
```

Esta es la función que se va a encargar de prender y apagar el pin del rele cada vez que se diga “uno”, “dos”, etc:

```
void action();
```

Declara la función “action” en la cual irán condicionales múltiples cuyas posibilidades de bifurcación (los “case”) serán los comandos grabados en el Easy VR con las respectivas acciones que se deben ejecutar al activarse cada comando.

```
void setup()
{
 // ---- Setup de los pin de los leds ---- //
 for(i=0 ;i<5 ;i++)
 {
    pinMode(leds[i], OUTPUT);
 }

 // ---- Setup del pin del rele ---- //
 pinMode(rele, OUTPUT);

 // ---- Setup del Easy VR ---- //
 if (bridge.check())
 {
  cli();
  bridge.loop(0, 1, 12, 13);
 }

 port.begin(9600);

 while(!easyvr.detect())
 {
  delay(100);
 }
```

```
easyvr.setPinOutput(EasyVR::IO1, LOW);
 prende(5);
 delay(200);
 parpadea();
 easyvr.setTimeout(5);
 easyvr.setLanguage(0);

 group = EasyVR::TRIGGER;
}
```

En todo programa de Arduino se debe utilizar la función “void setup” para informarle al compilador que pines van a ser usados como salida, cuales como entrada, cuál va a ser la velocidad de comunicación serial, etc.

```
// ---- Setup de los pin de los leds ---- //
 for(i=0 ;i<5 ;i++)
 {
    pinMode(leds[i], OUTPUT);
 }

 // ---- Setup del pin del rele ---- //
 pinMode(rele, OUTPUT);
```

Declara el arreglo de pines destinados a los leds y al pin conectado a la base del transistor que activa el relé como pines de salida.

```
if (bridge.check())
{
  cli();
  bridge.loop(0, 1, 12, 13);
 }
```

Llama a la función check perteneciente al objeto Easy Bridge, la cual es una función del tipo booleano que devuelve un verdadero si el modo puente debería ser iniciado. Si devuelve verdadero, se llama a la función cli() que se encarga de deshabilitar las interrupciones del Arduino.

```
port.begin(9600);
```

Inicia la comunicación serial entre Arduino y el Easy VR a 9600 baudios.

```
 while(!easyvr.detect())
 {
  delay(100);
 }
```

Es un bucle que verifica que el modulo esté listo para empezar a funcionar, ya que la función detect() perteneciente al objeto easyvr es una función booleana que es verdadera si el modulo satisface las condiciones para arrancar. Si detect() es falsa, se espera 100 milisegundos y se vuelve a preguntar si el modulo está listo, y así hasta que sea detectado.

```
 prende(5);
 delay(200);
 parpadea();
 easyvr.setPinOutput(EasyVR::IO1, LOW);
 easyvr.setTimeout(5);
 easyvr.setLanguage(0);
```

Si el modulo ha sido detectado exitosamente la función prende() y parpadea() hacen que los leds brillen de manera que estemos avisados que el modulo ha sido detectado. Luego, setPinOutput() configura el pin IO1 (pin donde se conecta el micrófono) del easy VR de manera que este en estado LOW. Por último, setTimeout(5) establece el umbral de confianza para utilizar el reconocimiento de los comandos de voz grabados en la memoria del módulo y setLanguage(0) establece el tiempo de espera que se utilizara para las tareas de reconocimiento.

```
void loop()
{
 group=GROUP_1;
 easyvr.setPinOutput(EasyVR::IO1, HIGH); // El Easy VR empieza a "escuchar"
 easyvr.recognizeCommand(group);

 do
 {
 }
 while (!easyvr.hasFinished());

 easyvr.setPinOutput(EasyVR::IO1, LOW); // LED off

 idx = easyvr.getWord();
 if (idx >= 0)
 {
  return;
 }
 idx = easyvr.getCommand();
 if (idx >= 0)
 {

  easyvr.playSound(0, EasyVR::VOL_FULL);

  action();
 }
 else
 {
  if (easyvr.isTimeout())
  {
   prende(1);
   apaga(1);
  }
  int16_t err = easyvr.getError();
  if (err >= 0)
  {
   prende(2);
   apaga(2);
  }
 }
```

La función loop() es donde se escribe el código que se encarga de la ejecución del programa (algo asi como el main() de una aplicación de consola en C).

```
group=GROUP_1;
```

Como se consta de un solo grupo de comandos y por defecto en el setup se inicializa la variable group en cero (group=trigger), valuamos la variable group (que es la que se le pasa al easy vr para que sepa a qué grupo de comandos pertenece la palabra que se va a decir).

```
easyvr.setPinOutput(EasyVR::IO1, HIGH);
```

Se configura el pin IO1 del Easy –VR como HIGH (prendido) de manera que el micrófono (que está conectado a este pin) comience a escuchar.

```
easyvr.recognizeCommand(group);
do
{
}while (!easyvr.hasFinished());
```

La función recognizeCommand(group) Inicia el reconocimiento de una palabra perteneciente al grupo “group”, tomando el audio que proviene del micrófono, pero los resultados de esta no están disponibles hasta que la función hasFinihed() no devuelve verdadero. Esta última, tomo el audio obtenido y devuelve entre 0-31 si el reconocimiento fue exitoso y -1 si no lo fue.

```
easyvr.setPinOutput(EasyVR::IO1, LOW);
```

Se apaga el pin IO1, por lo tanto el micrófono deja de escuchar (ya que en el código anterior se tomó audio y se lo analizo).

```
idx = easyvr.getWord();
```

La función getWord() obtiene el ultimo código de error que genero el easy vr, devolviendo entre 0 y 255 si hubo error , y -1 si no lo hubo.

```
if (idx >= 0)
{
   return;
}
```

Si getWord() devolvió un número mayor a -1 quiere decir que hubo algún error en el reconocimiento, por lo tanto se puede utilizar el return para analizar el error.

```
idx = easyvr.getCommand();
if (idx >= 0)
{
  easyvr.playSound(0, EasyVR::VOL_FULL);

  action();
}
```

La función getCommand() toma el audio obtenido del micrófono y procesado anteriormente, y reconoce si es concordante con algunos de los comandos grabados para el grupo en el que se está trabajando. Devuelve entre 0 y 31 si algún comando fue reconocido correctamente y -1 si no hubo coincidencias u ocurrió algún error. Este valor devuelto se almacena en la variable idx. Si idx es mayor que -1, o sea, que se reconoció exitosamente alguno de los comandos guardados en el Easy VR, de llama a la función acción l que tiene estructuras switch que primero se bifurcan por el grupo en el que se está trabajando (variable group) y luego por el comando de ese grupo que fue reconocido (variable idx).

```
else
{
  if (easyvr.isTimeout())
  {
    prende(1);
    apaga(1);
  }
  int16_t err = easyvr.getError();
  if (err >= 0)
  {
    prende(2);
    apaga(2);
  }
}
```

La función isTimeout() obtiene algún indicador de conflicto que se dio durante el reconocimiento, devuelve verdadero si hubo algún error en el tiempo de reconocimiento (por ejemplo nos e dijo nada o había mucho ruido), si algo de esto ocurrió, uno de los leds del gabinete parpadea.
La función getError() devuelve un número mayor a cero si algún error ocurrió en el procesamiento de la señal obtenida (por ejemplo, se obtuvo una palabra limpia pero no coincide con ninguno de los comandos grabados en el módulo). Si devuelve un número mayor a -1 (o sea que hubo algún error de los nombrados anteriormente) dos leds del gabinete parpadean:

```
void prende(int num)
{
  for(i=0 ;i<num ;i++)
  {
    digitalWrite(leds[i], HIGH);
    delay(80);
  }
}
```

Esta función se encarga de ir prendiendo uno por uno los leds.

```
void apaga(int num)
{
  for(i=0 ;i<num;i++)
  {
    digitalWrite(leds[i], LOW);
  }
}
```

Esta función apaga todos los leds independientemente de su estado.

```
void parpadea()
{
  for(j=0;j<2;j++)
  {
   for(i=0 ;i<5 ;i++)
   {
     digitalWrite(leds[i], HIGH);
   }
   delay(150);

   for(i=0 ;i<5 ;i++)
   {
     digitalWrite(leds[i], LOW);
   }
  }
}
```

Hace parpadear los leds con un tiempo de 150 milisegundos entre parpadeo.

```
void ReleParpadea(int num)
{
 if(estado==0)
 {
    for(i=0 ;i<num ;i++)
    {
        digitalWrite(rele, HIGH);
        delay(500);
        digitalWrite(rele, LOW);
        delay(500);
     }
  }
```

```
  else
  {
     for(i=0 ;i<num ;i++)
     {
         digitalWrite(rele, LOW);
         delay(500);
         digitalWrite(rele, HIGH);
         delay(500);
     }
  }
}
```

Esta función hace que el pin del relé se prenda y se apague el número de veces que se le pase por parámetro.

```
void action()
{
   switch (group)
   {
     case GROUP_1:
        parpadea();

      switch (idx)
      {
        case G1_HOLA:
            digitalWrite(rele, HIGH);
            estado=1;
          break;

                    case G1_UNO:
           ReleParpadea(1);
          break;

                    case G1_DOS:
           ReleParpadea(2);
          break;

                    case G1_TRES:
           ReleParpadea(3);
          break;

                    case G1_CUATRO:
           ReleParpadea(4);
          break;

                    case G1_CINCO:
           ReleParpadea(5);
          break;

                    case G1_SEIS:
           ReleParpadea(6);
          break;
```

```
                    case G1_CHAU:
                digitalWrite(rele, LOW);
                estado=0;
              break;

                    case G1_APLAUSO:
                if(estado==0)
                {
                  digitalWrite(rele, HIGH);
                  estado=1;
                }
                else
                {
                  digitalWrite(rele, LOW);
                  estado=0;
                }
              break;
            }
          break;
        }
      }
```

Esta función action() está formada por switchs que bifurcan la acción del programa según en qué grupo de comandos se esté trabajando y según que comando ha sido reconocido. Vale decir que si en el loop() se está trabajando con el grupo 2 y se dice un comando perteneciente a un grupo 1, este jamás será reconocido. Pero en este proyecto se trabajó con un solo grupo por lo cual no hay conflicto con eso.
Apenas se entra en el primer switch los leds parpadean indicando que se reconocio exitosamente un comando y que una acción va a realizarse. En el caso de que se hallan identificado los comandos uno, dos, tres, cuatro, cinco o seis, la lámpara parpadeara las veces correspondientes al número del comando (una, dos, tres, cuatro , cinco, seis veces). El comando HOLA prendera la lámpara cuando este apagada, pero en caso de estar prendida no hará nada. El comando CHAU apagara la lámpara en caso de que este prendida, pero en caso de estar apagada no hará nada. En cambio, el comando APLAUSO (que responde al sonido de dos aplausos, no a la palabra aplauso) prendera la lámpara en caso de que este apagada y la apagara si esta prendida.

Bibliografía

Apuntes:

- Manual de programación Arduino - José Manuel Ruiz Gutiérrez.
- Practicas con Arduino nivel 1 - José Manuel Ruiz Gutiérrez.
- EasyVR User Manual (Relase 3.4.2) – VeeaR.
- EasyVR: Guía rápida – Microsystems Engineering.
- Librería EasyVR para Arduino.

Enlaces:

- **http://carloscondor.com/a/Control-por-voz-sobre-un-Arduino.**
- **http://4imedio.blogspot.com.ar/2013/03/encender-leds-con-la-voz-easyvr.html**
- **http://arduiniana.org/libraries/newsoftserial/**
- **http://www.arduino.cc/**
- **http://www.veear.eu/**

Enlaces de Descarga:

- **Compilador Arduino:** **http://arduino.cc/en/Main/Software**
- **Controladores Easy VR:** **http://www.veear.eu/downloads/**

  Contiene manual de usuario, instalador del Easy voice commander (entrenador de comandos para el EasyVR), librerías para Arduino y más.
- **EasyVR guía rápida:**

  http://www.msebilbao.com/notas/downloads/EasyVR%20Guia%20rapida.pdf